Computer Science Fundamentals

Memory and processes

- RAM (Random Access Memory): Acts as the fast, volatile working space for the operating system and active apps. Data disappears when power turns off.
- ROM (Read-Only Memory) is non-volatile memory that stores essential instructions permanently. It holds system firmware and boot instructions.
- Memory Hierarchy: Ranks storage by speed and capacity, starting from ultra-fast CPU registers and cache, down to main RAM, and finally to slow, permanent storage like SSDs.
- Virtual Memory: Extends physical RAM using disk space, letting the system run large programs by swapping data in blocks called pages.
- Process Definition: A program loaded into memory and actively run by the CPU.
- Process Memory Layout: Splits into distinct segments: text (code), data (variables), stack (function calls), and heap (dynamic allocation).
- Multiplexing: Shares the CPU and memory safely across many running programs so they do not interfere with each other.
- A memory unit is a standard unit used to measure the amount of data that can be stored, processed, or transferred in a computer system. It represents the storage capacity of memory and storage devices.

Threads and Concurrency

- Process: An independent running program with its own isolated memory space, code, and system resources.
- Thread: A lightweight execution unit inside a process that shares the same memory space, heap, and file handles with other threads in that process. Each thread keeps its own stack, program counter, and registers.

- Concurrency: means dealing with multiple tasks at once by interleaving their execution (even on a single-core CPU via context switching).

Networking Fndamentals

- Computer networking fundamentals are the core rules, hardware, and concepts that allow devices to talk to each other and share data.

Core Network Components

- Nodes: Physical devices like computers, phones, or printers that send and receive data.
- Media: The wired cables (Ethernet, fiber optic) or wireless signals (Wi-Fi) that carry data.
- Switches: Devices that connect computers inside the same local network.
- Routers: Devices that connect multiple separate networks together and send data to the internet.
- Routers and switches control the flow of information.
- Wired media includes Ethernet and optical fiber cables.

Key Rules and Protocols

- Protocols: Predefined rules that let different devices understand each other.
- IP Address: A unique number given to every device on a network so data reaches the right place.
- TCP / UDP: Transport rules that manage how data packets move reliably or quickly.
- HTTP / DNS: Higher-level rules used to load web pages and translate website names into numbers.

How Computer Network Working

- Computer network operates by enabling devices to communicate and exchange data using a shared communication system. Each device in the

network follows predefined rules to ensure that data is transmitted accurately, efficiently, and securely.

- A network consists of nodes such as computers, servers, routers, and switches that send or receive data.
- These nodes are connected through links, which may be wired (cables, optical fiber) or wireless (Wi-Fi, radio signals).
- When data is sent, it is broken into small packets and transmitted across the network.

Cryptography basics

- In backend engineering, **cryptography is the foundation of data security**. It ensures that sensitive information remains confidential, unaltered, and verified.

1. The Core Pillars of Information Security

Before implementing cryptographic algorithms, you need to understand the **CIA Triad**, which defines security goals:

- **Confidentiality:** Keeping data hidden from unauthorized eyes (achieved via *Encryption*).
- **Integrity:** Ensuring data has not been altered or tampered with (achieved via *Hashing* and *MACs*).
- **Availability:** Ensuring authorized users can access the data when needed.
- **Authenticity:** Proving that data originated from a trusted source (achieved via *Digital Signatures*).

2. Hashing (One-Way Data Protection)

Hashing takes an input of any size and transforms it into a fixed-length string of characters. It is a **one-way function**, meaning it cannot be reversed.

- **How it works:** `Input -> Hash Function -> Output (Hash Value)`. You cannot go backward from the output to the input.

- **Key Properties:** Deterministic (same input always equals same output), fast to compute, and collision-resistant (two different inputs shouldn't produce the same output).

Backend Use Cases:

- **Password Storage:** Never store plain-text passwords. When a user creates an account, hash the password and store the hash. When they log in, hash the incoming password and compare it to the stored hash.
- **Data Integrity:** Verifying that a downloaded file hasn't been corrupted or altered.

What to use:

- **For Passwords:** Use slow, resource-intensive hashing algorithms designed to resist brute-force attacks: **Argon2id** (modern standard), **bcrypt**, or **scrypt**.
- **For Data Integrity:** Use **SHA-256** or **SHA-3**.
- *Avoid:* MD5 and SHA-1 (both are cryptographically broken and vulnerable to collisions).

---

3. Salting and Peppering (Securing Hashes)

If two users have the same password (e.g., `Password123`), their hashes will be identical. Attackers use precomputed lists of hashes called **Rainbow Tables** to crack them.

- **Salting:** Adding a unique, random string of characters to each password *before* hashing it. The salt is stored in the database alongside the hash. This ensures identical passwords result in completely different hashes.
- **Peppering:** Adding a secret string (a pepper) to the password before hashing. Unlike the salt, the pepper is **not** stored in the database; it is typically kept in an environment variable or a hardware security module (HSM).

---

4. Symmetric Encryption (Shared Secret)

Symmetric encryption uses the **same secret key** to both encrypt (lock) and decrypt (unlock) data.

- **Pros:** Extremely fast and efficient. Ideal for large amounts of data.

- **Cons:** Key distribution problem. Both parties must securely share the key beforehand. If an attacker steals the key, all data is exposed.

Backend Use Cases:

- **Database Encryption (Data at Rest):** Encrypting sensitive columns like credit card numbers, SSNs, or PII (Personally Identifiable Information).
- **Session Tokens:** Encrypting local session data before sending it to a client.

What to use:

- **AES-GCM (Advanced Encryption Standard in Galois/Counter Mode):** This is an *Authenticated Encryption* scheme (AEAD). It not only encrypts the data but also guarantees its integrity (tamper-proof).

---

5. Asymmetric Encryption (Public-Private Key Pairs)

Asymmetric encryption uses a mathematically linked pair of keys: a **Public Key** and a **Private Key**.

- **Public Key:** Can be shared freely with anyone. Used to *encrypt* data.
- **Private Key:** Kept strictly secret by the owner. Used to *decrypt* data.
- *Rule:* What one key locks, only the other can unlock.

Backend Use Cases:

- **Secure Connections (Data in Transit):** Establishing HTTPS/TLS connections. The public key encrypts a temporary symmetric key, which is sent over the network, and the private key decrypts it.
- **Digital Signatures:** The private key signs a piece of data (like a JWT or an API response), and anyone with the public key can verify that the data actually came from the server and wasn't altered.

What to use:

- **RSA** (2048-bit or 4096-bit keys)
- **ECC (Elliptic Curve Cryptography):** Modern preferred choice (e.g., Ed25519). It offers the same level of security as RSA but with much smaller key sizes, resulting in faster computations and less network overhead.